Circular Grid

Technical Report: System Architecture and Data Handling

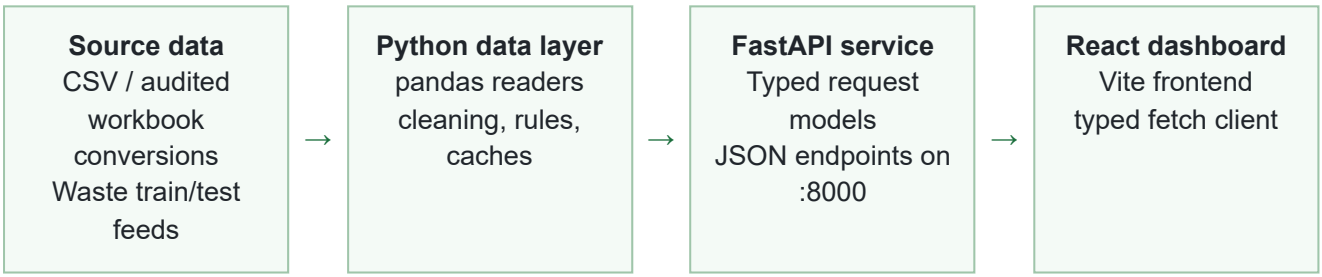
> **Scope.** This report describes the implemented local dashboard system, its service boundaries, data flow, storage conventions, transformation rules, and operational limitations. It is based on the repository configuration, API, scripts, and frontend contract.

1. System purpose

Circular Grid is a campus resource dashboard for eight buildings. It combines hourly electricity, water, and footfall streams with waste-event data. The user-facing product exposes campus overview, building forecasts, waste allocation, classifier activity, connectivity, data-quality, leaderboard, and cost/scale views.

The current deployment is intentionally lightweight: the API reads prepared CSV files directly rather than requiring a database or external cloud service. This keeps local demonstration and replay simple while preserving clear seams for later model and storage integration.

2. Logical architecture



Supporting modules sit beside the API rather than inside a distributed runtime: ingestion/audit handles source validation; edge buffering and reconciliation model intermittent connectivity; forecasting and anomaly modules provide analytical building blocks; classifier training evaluates waste models. The API currently contains the production-facing read path and a deterministic rule classifier.

3. Runtime components

Component	Responsibility	Boundary
FastAPI + Uvicorn	HTTP routing, CORS, validation, status/error responses	`src/api/main.py`
pandas data access	Read, normalize, filter, aggregate CSV data	`data/*.csv`
React/Vite	Dashboard pages and typed API consumption	`frontend/src`
Configuration	Paths, building metadata, units, thresholds, horizons	`config.py`

> **Deployment profile:** the documented local run binds the API to localhost:8000 and the frontend dev server to localhost:5173. CORS is restricted to those two frontend origins.

4. Data sources and lifecycle

Configuration separates immutable raw workbooks from derived data. The upload directory is controlled by `CIRCULAR_GRID_UPLOAD_DIR`; configured workbook names are mapped to canonical datasets for buildings, calendar, daily summary, hourly streams, and manual logs. The data workflow converts and audits supplied workbooks, while the explicit demo-data target generates synthetic waste events only when requested.



5. Canonical data handling

Dataset	Use	Handling in API
`hourly_streams_clean.csv` (fallback: hourly_streams.csv)	Building time series	Column names are lower-cased; timestamp is selected from `sampled_hour` or `timestamp`; invalid timestamps are dropped; rows are sorted.
`waste_events_train.csv`	Waste allocation	Weight is coerced to numeric, missing values become zero, then grams are converted to kilograms.
`waste_events_test.csv`	Classifier feed	First 50 events are classified by sensor-rule inference; this unlabeled feed is not an accuracy evaluation.
`models/classifier_results.json`	Optional model metrics	Loaded when present; otherwise the API returns an explicit rule-based model entry with zero evaluation metrics.

Required-file failures are surfaced as HTTP 503 responses. Unknown buildings, streams, and forecast horizons are rejected with 404 or 400 responses. Data values pass through a numeric safety helper that converts missing or non-finite values to zero at response boundaries.

6. Transformations and business rules

- **Building overview:** the API takes the latest 24 rows per building, exposes current electricity/water/footfall, and emits an electricity sparkline. A building is online when its last timestamp is within two hours of the dataset latest timestamp; otherwise it is stale.
- **Forecast response:** recent history is returned as actual values. Future values use the latest 24-hour mean, with lower and upper bounds at 80% and 120% of the baseline.
- **Waste allocation:** events are grouped by building and class. Organic waste uses the configured 4:1 compost ratio; recyclable classes are separated from landfill; diversion is the non-landfill share.
- **Classification:** moisture, metal signal, transparency, acoustic hardness, and weight drive deterministic class rules with fixed confidence values. Missing sensor values are normalized before inference.

7. Data quality and reproducibility

Configuration centralizes units, date bounds, building identifiers, thresholds, and random seed. The pipeline can therefore be replayed with the same settings. The frontend types document response shape and estimated/status semantics, reducing drift between API and UI.

8. API and presentation contract

The frontend uses a single typed client with `/api` as its base path. GET requests cover campus overview, building forecast, anomalies, waste allocation, classifier feed, connectivity, data quality, leaderboard, and cost/scale. A POST endpoint accepts classifier sensor values and returns predicted class, confidence, and measured inference latency.

Surface	Data contract	Operational meaning
Overview	Buildings, status, current metrics, sparkline	Latest campus state and freshness
Forecast	Actual/forecast/lower/upper series	Short-horizon planning for 6 or 12 hours
Waste	Generated, composted, recycled, landfill, diversion	Material flow and landfill avoidance
Classifier	Events, model metadata, prediction response	Segregation assistance and feed visibility
Connectivity/quality	Status, last seen, quality counters	Trust and operational monitoring

9. Reliability, limitations, and extension points

Implemented safeguards

- Lazy CSV reads are cached with a one-entry cache, avoiding repeated disk parsing during a process lifetime.
- Timestamp coercion, numeric coercion, missing-file errors, parameter validation, and localhost-only CORS provide explicit boundaries.
- Configuration distinguishes measured source data from optional synthetic demo data. The README warns that generated waste events must not be reported as measured results.

Current limitations

- Several dashboard endpoints are scaffolds: anomalies, data quality, leaderboard, and cost/scale currently return empty, zero, or fixed values rather than live analytical results.
- The forecast endpoint uses a mean baseline and fixed intervals; it is a service contract, not a trained probabilistic forecast.
- Connectivity is inferred from file timestamps and does not yet perform live edge reconciliation. The edge modules provide the design foundation for buffered/offline operation.
- CSV storage and process-local caching are suitable for a local dashboard but do not provide transactional writes, concurrent ingestion, retention policies, or multi-user isolation.

10. Recommended production evolution

Keep the current API contracts stable while replacing the file reader with a versioned storage adapter. Add schema validation and lineage IDs at ingestion, persist quality metrics instead of returning placeholders, and move forecasts/classifier inference behind explicit model interfaces. For scale, use append-only event storage plus materialized hourly aggregates, background refresh, authentication, and observability for freshness, latency, missing data, and model drift.

> **Conclusion:** Circular Grid has a clear, testable local architecture: audited/derived files feed a typed FastAPI boundary and a React dashboard. Its strongest current property is transparent data handling; its main next step is replacing placeholder analytics and process-local CSV access with durable, observable production services.